



DEMAND → BED

Decision intelligence for resident-inventory matching

SYNTHETIC PROTOTYPE · NOT STANZA INTERNAL DATA

PRODUCT CASE STUDY · AI-ENABLED DECISION SYSTEM

Turning fragmented leasing demand into one explainable, resident-first decision.

Demand-to-Bed is a prototype decision system for matching accommodation leads to available inventory — built to show how I frame an ambiguous, multi-objective product problem, not just how I ship a feature.

25,000

SYNTHETIC LEADS

150

PROPERTIES

112,394

LOGGED INTERACTIONS

11,013

SALES ACTIONS

THE PROBLEM

Resident demand, bed inventory and sales capacity are tracked separately, so every match is reassembled by hand under changing constraints. A recommendation that only ranks "best property" ignores whether that property is actually available, appropriate, or commercially sustainable to sell.

THE INSIGHT

Property recommendation is not a single-objective ranking problem. It's a constrained, multi-objective decision: resident eligibility has to gate the options before booking probability, inventory pressure or contribution are allowed to influence order.

WHAT I BUILT

A full-stack decision engine — eligibility gate, explainable resident-fit scoring, a booking-propensity model, transparent inventory pressure, a configurable decision policy, next-best-action logic, and a human accept/modify/reject loop — wired to a live product UI across sales and resident surfaces.

KEY PM DECISIONS

Hard constraints before optimization · transparent, editable policy weights · explainable recommendations · human override captured as signal · every score resolves to an action · synthetic results never dressed up as real ones.

Demand, supply and sales were each solving a different problem.

Stanza's public product already lets someone browse, shortlist and book a tour. The harder problem sits beneath that: deciding, for one resident with real constraints, which of 150 properties the business should put in front of them — and what a salesperson should do next.

DEMAND

What the resident brings

City, locality, budget, room type, move-in date, stay length, and a primary driver — commute, food, safety, community, amenities — plus an intent score from views, calls and WhatsApp activity.

SUPPLY

What inventory can offer

Occupancy, available beds, days vacant and expected move-outs, refreshed daily per property — the gap between a property that looks good on paper and one that can house someone this month.

BUSINESS

What the match needs to do

Booking conversion and contribution per booking, weighed by a sales team already logging 11,013 accept / modify / reject decisions against the leads they work.

Why a naive recommender breaks down

A model that simply ranks "most attractive property" will happily surface one that's over budget, in the wrong city, or fully occupied — attractiveness and eligibility are different questions. It will also rank commercially convenient inventory above a better resident fit, because nothing distinguishes a hard requirement from a soft preference.

Collapse eligibility, fit, conversion and inventory pressure into one undifferentiated score, and none of them can be inspected, defended, or corrected — which is the position sales was already in.

The question the product has to answer

"Among the properties this resident is genuinely eligible for, which should we recommend right now — given their needs, real availability, booking likelihood, and this week's business priorities?"

Everything on the pages that follow — eligibility, fit, propensity, inventory pressure, the policy layer — exists to answer that sentence without collapsing it into one opaque number.

Where the tension shows up

RESIDENT CONSTRAINT

"Private room, under ₹18k, Gurgaon, in 2 weeks."

VS

BUSINESS PRESSURE

"Property X has 20 empty beds and aging vacancy."

→

PRODUCT PRINCIPLE

Inventory pressure can reorder eligible options — it can never make an ineligible one eligible.

WHO FEELS THIS FIRST

The sales rep, mid-conversation, holding a resident's requirement and a dashboard that don't talk to each other.

WHAT IT COSTS TODAY

Time spent reconciling constraints by hand, and recommendations a resident has to trust without seeing why.

WHAT'S ON THE NEXT PAGE

Three findings from the supplied dataset that shaped how I answered this, in order.

The dataset doesn't point to one broken step — it points to fragmentation.

The synthetic portfolio pairs demand, supply and sales-outcome data at a scale that lets each side of the marketplace be inspected before being recombined. Three findings shaped the decisions on the pages that follow.

OBSERVATION • 01

Pressure is hyper-local, not portfolio-wide

Bangalore • 17.1 leads / Triple rooms available bed



Hyderabad • 1.7 leads / Triple rooms available bed



Same room type, two cities: Bangalore Triple rooms carry 840 leads against 49 available beds; Hyderabad Triple rooms carry 449 leads against 268 beds.

Implication: inventory intelligence must work at the property/segment level — a portfolio-wide push would misfire in one of these two cities.

OBSERVATION • 02

Rejections aren't concentrated in one cause

Other / uncategorized 1,039



Delayed follow-up 849



Room type mismatch 836



Price 829



Changed plans 822



Commute 816



8,027 of 11,013 sales actions were rejections, spread almost evenly across room type, price, plan changes and commute — no reason clears 13%.

Implication: sales isn't failing at one step. They reconcile several shifting constraints at once — the product should support that, not just rank.

OBSERVATION • 03

"Lost" is nearly 1.5× the size of "Booked"

13,987

OPEN

6,561

LOST

4,452

BOOKED

Of 25,000 leads, booking conversion lands at 17.8%, but over a quarter of demand exits as "Lost" — more than the group that converts.

Implication: follow-up timing and next-best-action matter as much as ranking quality for moving the north star.

This is also why the product records *why* a recommendation was accepted, modified or rejected, not just whether it was — see page 08.

From observation to product insight

None of these findings are about model accuracy. They're about where fragmentation costs money: uneven pressure across near-identical inventory, a sales team reconciling many constraints at once, and a demand pool where losing the lead is more common than winning it. That reframes the brief from *"rank properties better"* to *"give the person deciding the constraints, the context, and a next step, together."*

SCALE OF THE SOURCE DATA

25,000 leads • 150 properties • 18,000 daily inventory snapshots • 112,394 interactions, all synthetic and disclosed as such throughout this build.

HOW I VERIFIED THESE NUMBERS

Recomputed directly from the CSVs in the repository, not taken from a README — the same figures the running product's API returns.

WHAT'S ON THE NEXT PAGE

Who's actually making this decision, and the operating principles that follow from these three findings.

Two people make this decision. The system has to work for both.

A sales rep and a resident are asking versions of the same question from opposite sides of the transaction. Revenue and product operations sit behind both, watching for where demand and supply stop lining up.

PRIMARY • SALES / GROWTH

“What should I do with this lead, right now?”

JOB TO BE DONE

Reconstructing eligibility, fit and inventory by hand for every lead, then defending the recommendation when it doesn't match commercial pressure.

DECISION THEY MAKE

Which eligible property to present first, and whether to schedule a visit, follow up, or offer an alternative.

SUCCESS LOOKS LIKE

A defensible recommendation with visible reasons, and a fast way to accept, modify or reject it.

PRIMARY • RESIDENT

“Which home genuinely fits me?”

JOB TO BE DONE

Sifting listings without knowing which ones are actually available for their move-in date, budget and room type.

DECISION THEY MAKE

Whether to shortlist, and whether to commit to a visit.

SUCCESS LOOKS LIKE

Options that are already known to fit, with a clear reason for each, and an easy path to book a visit.

SECONDARY • PRODUCT / REVENUE

“Where is demand and inventory misaligned?”

JOB TO BE DONE

Occupancy dashboards show *what* is vacant, not *why*, or which lever (pricing, routing, incentive) would help.

DECISION THEY MAKE

Where to intervene — pricing, sales routing, or incentives — before inventory ages further.

SUCCESS LOOKS LIKE

A segmentation that separates properties worth protecting from ones that need active intervention.

Product principles

01 Resident fit first, business optimization second.

Inventory pressure and contribution can reorder eligible options, but they can never promote an incompatible property into contention. Business needs adjust the order; they don't override the requirement.

02 Hard constraints are gates, not weighted preferences.

City, room type, budget and near-term availability are pass/fail checks that run before any scoring happens — not soft factors that a high enough business score could outweigh.

03 Every recommendation must be explainable.

A ranked property without a stated reason is not decision support; it's a black box wearing a decision's clothing. Reasons are computed from the same fit components a person could audit.

04 AI recommends; humans accept, modify, or reject.

The system proposes; a person decides. This keeps the model's real-world weaknesses (below) from becoming unreviewed business decisions.

05 Feedback must become a learning signal, not a dead end.

A modified or rejected recommendation is evidence the policy or the data was wrong somewhere — it's captured, not discarded, so the product can be corrected.

06 Synthetic assumptions must never be represented as internal metrics.

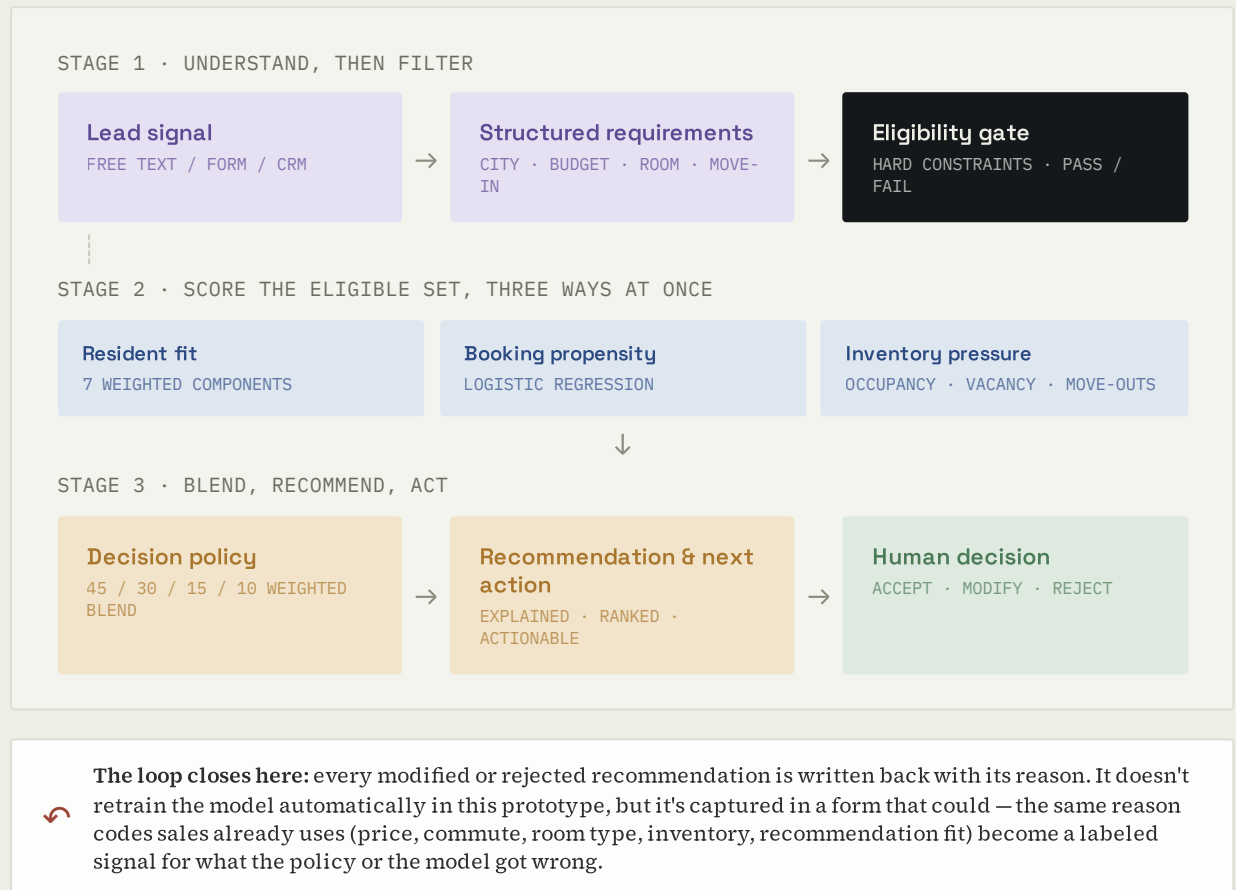
Every screen, endpoint and document in this build discloses that the data and results are synthetic, so the prototype can never be mistaken for validated company performance.



The next page turns these six principles into the actual pipeline — the specific order eligibility, fit, propensity, inventory pressure and policy run in, and what each stage is, and isn't, allowed to decide.

One pipeline, from a lead's message to a scheduled visit.

Every recommendation moves through the same sequence, whether it starts on the sales side or in resident mode. Nothing after the eligibility gate can promote a property that failed it — later stages can only reorder what already passed.



What runs per property, per lead

Fit, propensity and pressure are computed fresh for every eligible property against the specific lead — there's no cached "best property" independent of who's asking.

What's configurable, no code change

The decision-policy weights and resident-fit component weights are runtime settings, editable from the product's Settings screen and validated to sum to 100%.

What never gets skipped

City, room type, budget and near-term availability. A property failing any one of these never enters scoring, however attractive its other numbers are.

Reading the pipeline against the six principles

STAGE	PRINCIPLE IT IMPLEMENTS
Eligibility gate	Hard constraints are gates, not weighted preferences — and resident fit is checked before any business signal.
Fit / propensity / pressure, in parallel	Three independent, inspectable signals — not one blended score computed in a single opaque step.
Decision policy & recommendation	Every recommendation is explainable, and the weights producing it are visible and configurable.
Human decision	AI recommends; a person accepts, modifies, or rejects — and that choice becomes feedback, not a dead end.

A property becomes eligible before it becomes attractive.

For a resident who needs a private single room in Gurgaon, under ₹18,000, moving in two weeks — the gate runs first, on real properties from the portfolio:

P107 Stanza Residence 107 Sector 52 • Single • ₹16,981/mo ✓ City, room, budget and availability all pass	P063 Stanza Residence 063 Golf Course Road • Single • ₹27,417/mo ✗ Above budget (₹27,417 vs ₹18,000 max)	P111 Stanza Residence 111 Sector 52 • Double • ₹12,107/mo ✗ Room type mismatch (Double vs Single required)
---	---	---

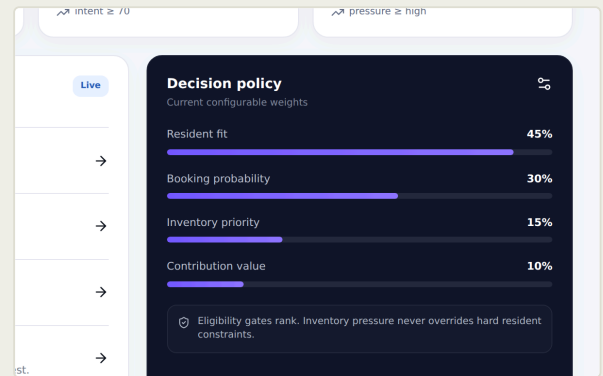
P063 is a stronger property by rating and P111 sits in the same locality — neither can out-score P107 on business terms, because neither is in the eligible set to begin with. **City, room type, budget, and near-term availability are pass/fail checks, not weighted preferences.** Only what clears all four is scored at all.

The decision policy, after the gate

Once a set of properties is eligible, the decision score blends four signals at prototype weights that are visible in the product itself, not buried in code:

SIGNAL	WEIGHT	SOURCE
Resident fit	45%	7-component match score
Booking probability	30%	Propensity model × property context
Inventory priority	15%	Occupancy, vacancy, move-outs
Contribution value	10%	Booking value vs. portfolio p95

These weights, and the seven resident-fit component weights beneath them, are editable at runtime from the product's Settings screen — a policy change is a configuration change, not a deploy.



Command Center • Decision policy panel

Real product screenshot

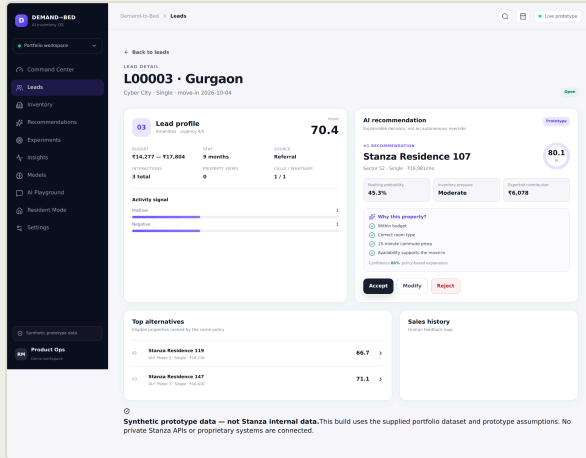
Inside "resident fit": seven components

LOCATION • 20%	BUDGET • 20%
ROOM TYPE • 15%	COMMUTE • 15%
AVAILABILITY • 10%	AMENITIES • 10%
PREFERENCES • 10%	

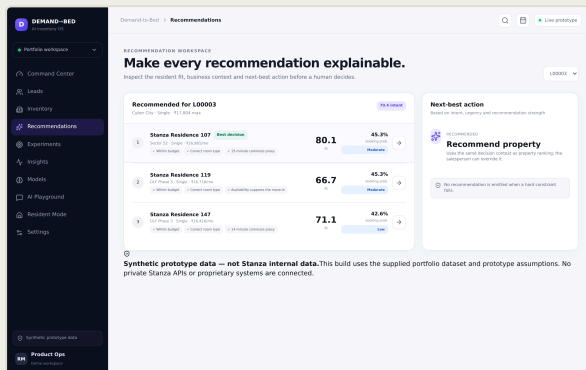
Location and budget carry the most weight because they're the constraints residents state first and relax last, based on the sales-action reasons on page 03.

A ranked property without a stated reason is not decision support.

The lead workspace doesn't hand sales a number. It hands them a property, a fit score, a booking probability, an inventory read, and the specific reasons — computed from the same components a person could check by hand.



Lead detail • L00003, Gurgaon Real product screenshot



Recommendation workspace • ranked alternatives Real product screenshot

Explainability, concretely

Every "why" line on the card — within budget, correct room type, strong locality fit, availability supports the move-in — is read directly from that property's fit components, not generated after the fact. A salesperson can trace a recommendation back to a fact about the property, and dispute it if the fact is wrong.

That matters for four reasons: it lets a rep defend the recommendation to a resident, it lets a reviewer audit whether the policy is behaving as intended, it turns a rejection into a specific disagreement instead of a vague override, and it means a bug in the reasoning is visible instead of buried in a score.

Next-best-action logic

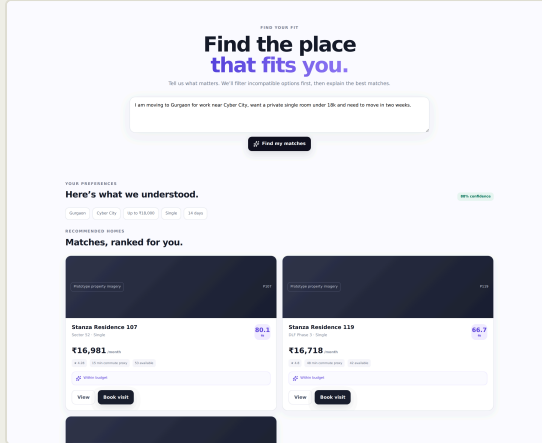
The top recommendation resolves to one action, evaluated in order:

- Booking probability $\geq 70\%$ AND urgency $\geq 4/5$ SCHEDULE VISIT
- Booking probability $\geq 55\%$ PERSONALIZED FOLLOW-UP
- Inventory pressure ≥ 75 AND resident fit ≥ 78 RECOMMEND PROPERTY
- Resident fit < 70 PRESENT ALTERNATIVE
- Otherwise RECOMMEND PROPERTY

Design note: the rules run in a fixed priority order — booking likelihood and urgency are checked before inventory pressure ever gets a vote. A property under commercial pressure can move up the ranking; it cannot invent urgency that isn't there.

The same engine, two surfaces — and a human always closes the loop.

Resident Mode runs the identical eligibility → fit → propensity → ranking pipeline as the sales workspace, in plain language. On the sales side, every recommendation still ends with a person: accept, modify, or reject.



Resident Mode • matches for a Gurgaon, ₹18k, 2-week request

Real product screenshot

Same requirement as the eligibility example on the previous page, same underlying ranking — translated into a resident-facing card with a fit score, price, commute proxy and a one-line reason, plus a direct path to book a visit.

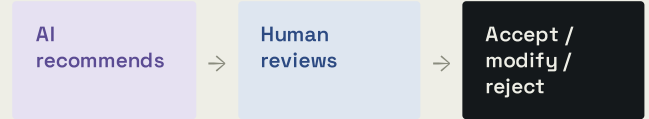
Why residents see the same ranking sales does

A separate, friendlier-looking resident algorithm would be easy to build and dishonest in practice — if the eligibility gate or the fit weights are ever wrong, both surfaces should be wrong the same way, visibly, rather than drifting into two different products that happen to share a brand.

What's resident-specific

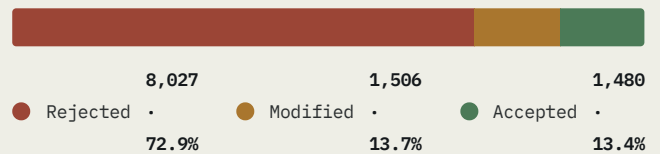
Only the surface changes: plain-language input instead of structured fields, a photo-card layout instead of a data table, and a booking action instead of a sales action. The eligibility gate, fit score and ranking logic underneath are identical.

Accept, modify, reject — captured, not discarded



The decision and its reason are written to the product's own store — the same 11,013-row shape as the historical sales log — so a modification is data, not a dead end.

What the historical log already shows

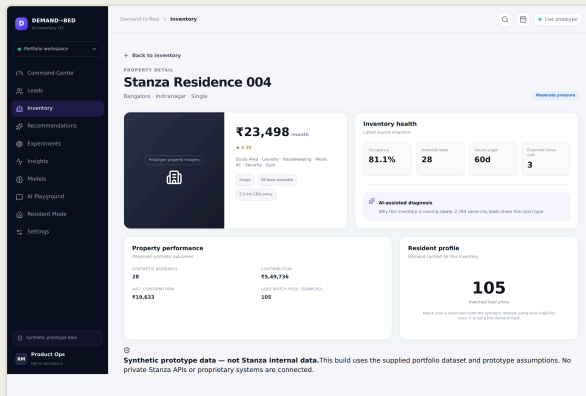


73% of logged sales actions were rejections. Read uncharitably, that looks like a broken recommender. Read against page 3's finding — rejection reasons spread evenly across price, room type, commute and availability — it looks like a sales team correctly refusing to force weak matches, exactly the behavior eligibility gating is meant to formalize rather than override.

Why this matters for the roadmap: a policy or model that pushed acceptance rate up without narrowing *why* people reject would be optimizing the wrong metric. Override reason, not override rate alone, is the signal worth tracking.

Inventory pressure explains why a property is moving slowly — and AI is scoped tightly around it.

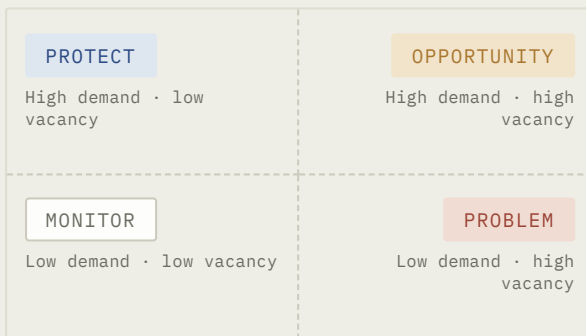
Occupancy alone tells operations *what* is vacant. Pressure — occupancy, available beds, vacancy age and expected move-outs, in a transparent formula — segments the portfolio into where to protect, where to push, and where to just watch.



Property detail • AI-assisted diagnosis

Real product screenshot

Demand-inventory segmentation



Where AI sits in the stack

Language understanding is deliberately separated from decision policy. The model that could be wrong sits behind rules that can't bend, and a person sits after both.

Language understanding

OPTIONAL LLM, DETERMINISTIC FALLBACK

free text → city budget room type

move-in horizon

Deterministic rules

ALWAYS RUNS, NEVER LEARNED

eligibility gate policy weights

next-best-action thresholds

Statistical model

LOGISTIC REGRESSION, ONE SIGNAL AMONG FOUR

booking propensity 6 lead-level features

adjusted by property fit

Human judgment

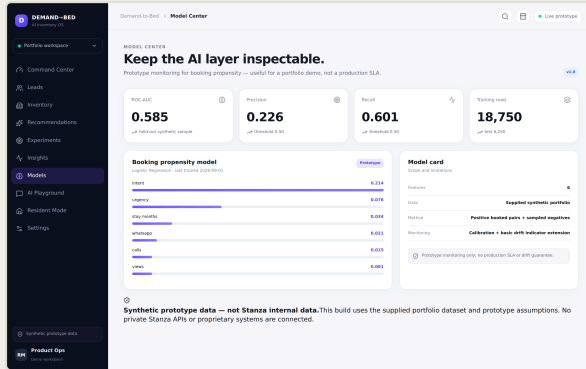
ALWAYS THE LAST STEP

accept modify reject reason capture

Why this ordering, not an end-to-end model: an LLM guessing at "which property" would be unauditible and occasionally confidently wrong. Splitting extraction from policy means the part that's allowed to be probabilistic (reading intent from text) never touches the part that has to be defensible (which properties are even eligible).

Before shipping on uplift, I'd want three more checks — and the model card says so.

Both the propensity model and the ranking experiment are visible in the product exactly as they perform, not as I'd want them to perform. I reran the training myself to confirm the numbers below.



Model Center · held-out synthetic metrics

Real product screenshot, independently reproduced

0.585

ROC-AUC

0.226

PRECISION

0.601

RECALL

Read honestly: a logistic regression on six lead-level engagement features, trained on 18,750 rows and tested on 6,250, is a usable prototype ranking signal — not a validated predictor of who books. The model card in the repository says exactly this, and the UI never claims otherwise.

What I'd improve before production

Property-level labels

Current target is lead-level Booked/Not-Booked. A true lead-property propensity needs negative sampling against properties actually shown, not just the outcome.

Temporal validation

The 25/75 split is random, not time-based — a production model needs to prove it generalizes forward, not just across a shuffled sample.

Calibration

Recall (0.601) is usable for ranking; precision (0.226) means raw scores shouldn't be read as real probabilities without calibration.

Leakage review

Engagement features (calls, views, WhatsApp) can move after a booking decision is already effectively made — worth auditing for reverse causality.

Offline ranking metrics

AUC on a binary outcome doesn't tell us if the top-3 ranked properties per lead are the right top-3 — needs NDCG or similar.

Monitoring in production

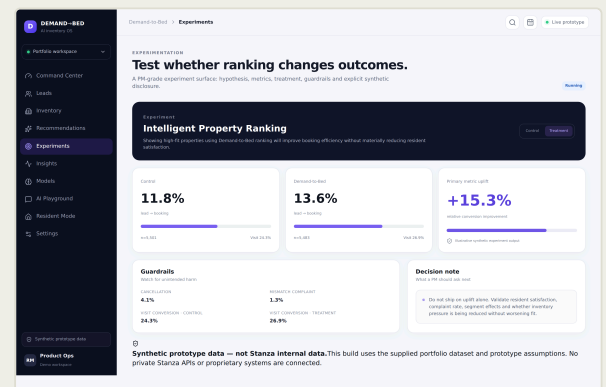
This build computes metrics once at startup. A shipped version needs drift detection and a retraining trigger.

The ranking experiment, disclosed as synthetic

Hypothesis: showing high-fit properties via Demand-to-Bed ranking improves booking efficiency without materially reducing resident satisfaction.

Control: default ordering, n=5,501 · Treatment: Demand-to-Bed ranking, n=5,483. Result: 11.8% → 13.6% conversion, guardrails (cancellation 4.1%, mismatch complaint 1.3%) held steady.

This is a hard-coded, illustrative output, not a real online experiment — the product labels it that way in the UI itself, and this document repeats that label rather than presenting +15.3% as a measured result.



Experiments · synthetic disclosure shown on-card

Real product screenshot

What I'd measure, and what could quietly go wrong.

A north star and a guardrail set, plus the risks I'd actually watch for before trusting this system with real commercial decisions.

NORTH STAR

**Qualified lead →
booking conversion**

DRIVERS

Eligible-property coverage ·
recommendation acceptance rate ·
visit conversion · sales response time

GUARDRAILS

Cancellations · mismatch complaints
· override rate by segment · vacant-
bed days

Secondary business outcomes — vacancy reduction, contribution, inventory utilization — sit behind the north star; none of them are allowed to move the guardrails in the wrong direction unnoticed.

Risks I'd actually watch

RISK	WHY IT MATTERS	MITIGATION / SIGNAL
Stale inventory	Beds shown as available may have moved by the time a visit happens; the daily snapshot isn't a live feed.	Surface snapshot date on every screen (already done); shorten refresh cycle before any real booking commitment.
Weak propensity labels	A 0.226-precision signal, over-trusted, produces confidently wrong prioritization.	Keep propensity as one of four blended signals, never the sole ranking key; recalibrate before raising its policy weight.
Over-optimizing contribution	A policy tuned purely for revenue can quietly erode resident fit within the eligible set.	Resident fit carries the largest weight (45%) by design; guardrail on mismatch complaints before ever raising contribution's.
Sales over-trust	An explainable UI can still be trusted more than a 0.585-AUC model deserves.	Model Center exposes the real metrics in-product; case study and onboarding should say the same thing sales sees.
Feedback loop bias	If overrides aren't reviewed, the policy only ever hears from residents sales chose to show, not the full pool.	Track override rate by segment, not just in aggregate, so a blind spot in one city or room type stays visible.
Synthetic-data limits	Every pattern in this case study is a property of the generator, not of real resident behavior.	Every screen, endpoint and page of this document discloses synthetic status; no figure here is presented as validated.

Trade-offs I made

Resident fit vs. inventory optimization: fit carries 3× the weight of inventory priority. Slower vacancy clearing was an accepted cost of not overriding real requirements.

Automation vs. human control: every recommendation is reviewable and reversible. That's slower than autonomous booking and deliberately so, given the model's real precision.

Immediate value vs. production readiness: the propensity model, inventory formula and experiment are all prototype-grade by design — validating the product concept mattered more than production-hardening a single component early.

What I'd hold back, where this goes next, and what building it demonstrates.

What I would not build yet

Prioritization is also about what a v1 doesn't need:

- Fully autonomous booking without human review
- Real-time pricing optimization
- Multi-agent LLM orchestration for ranking itself
- Cross-property portfolio-wide auto-incentives

Why hold these back

Each one assumes a maturity the current build doesn't have yet — live inventory, a validated propensity model, or a policy governance process. Building them now would add surface area without adding trust.

Roadmap

PHASE 1

Decision Support MVP

WHAT'S BUILT HERE

- Eligibility, fit, propensity, pressure
- Explainable ranking + next action
- Human accept / modify / reject

PHASE 2

Learning & Workflow Integration

NEXT

- CRM / telephony integration
- Override reasons retrain the policy
- Property-level propensity labels

PHASE 3

Real-Time Decision Intelligence

LATER

- Live availability feed
- Governed policy versioning + approvals
- Causal experimentation platform

What this project demonstrates

Problem framing Reframed “recommend a property” as a constrained, multi-objective decision before writing any ranking logic.	Constraint-based thinking Treated eligibility as a gate, not a weighted input — the single highest-leverage decision in the build.
Data-driven discovery Read the synthetic dataset for where fragmentation actually costs money, rather than assuming the problem.	AI product judgment Scoped the LLM to language understanding only; kept policy and eligibility deterministic and auditable.
Honest evaluation Reproduced the model's real metrics myself and reported them as a prototype signal, not a finished predictor.	Workflow design Resolved every score to a next action, and every human override to a captured, reusable signal.

A note on rigor: building this meant running it, not just reading it. Reproducing the model metrics surfaced no surprises — the repository's own numbers held. Actually running the frontend did surface two real defects (a duplicate component declaration that broke the build, and a layout bug that nested two full-bleed screens inside the internal app shell); both are fixed in the build these screenshots come from. I'd rather show that than a clean repository I hadn't actually tested.

PROJECT	ROLE	INSIGHT	EVIDENCE
Demand-to-Bed	Product manager & builder, end to end	Property recommendation is a constrained, multi-objective decision, not a single ranking problem.	Every figure in this document is reproducible from the repository's own data and code — nothing here is asserted without a source.